

Lecture 16

Neural Networks and PyTorch

Introduction to neural networks and PyTorch

Reference Book Chapters: Chapter 6 up to section 6.3.1

Also Example from Ch6 GoodFellow et al. Deep Learning Book:
<https://www.deeplearningbook.org/contents/mlp.html>

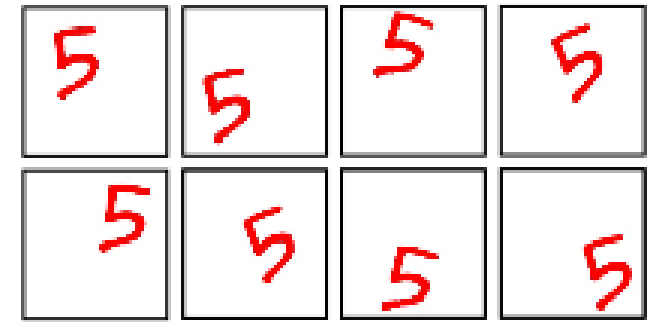
EECS 189/289, Spring 2026 @ UC Berkeley

Agenda

- **Neural Networks**
 - **Neurons**
 - **Feed-Forward Neural Network**
- **Universality Theorem**
- **Activation Functions**
- **Building & Training Neural Networks in PyTorch**

Data Manifolds

- Real-world high-D data (images, audio, text) usually lives on a **low-dimensional manifold** inside the huge input space.
 - 64×64 images are 4096 dimensional vectors, but a digit's pose/position/orientation $\approx$ a few DoF
- Fixed bases such as polynomials explode with dimension.



If we instead model **along the manifold**, complexity scales with manifold dimension (small), not data dimension (huge). We need **representation learning**: extracting features from raw data that are then used to understand images, text etc.

Data Manifolds → Neural Networks

If we instead model **along the manifold**, complexity scales with manifold dimension (small), not data dimension (huge).

Neural networks = data-dependent bases

- NNs **learn basis functions from data** that adapt to the manifold (vs. hand-crafted/fixed).
- They also learn which **directions on the manifold are relevant** to the task.
- I.e. they perform representation learning.

Neural Networks

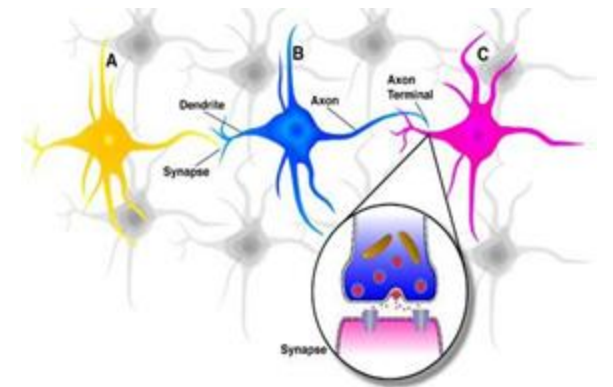
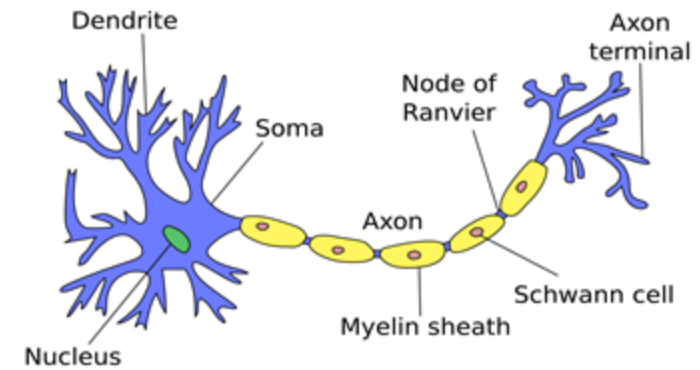
- **Limitations of Fixed Basis Functions**
- **Neural Networks**
 - Neurons
 - Feed-Forward Neural Network
- Universality Theorem
- Activation Functions
- Building & Training Neural Networks in PyTorch

Neurons

- **Neural Networks**
 - **Neurons**
 - Feed-Forward Neural Network
- Universality Theorem
- Activation Functions
- Building & Training Neural Networks in PyTorch

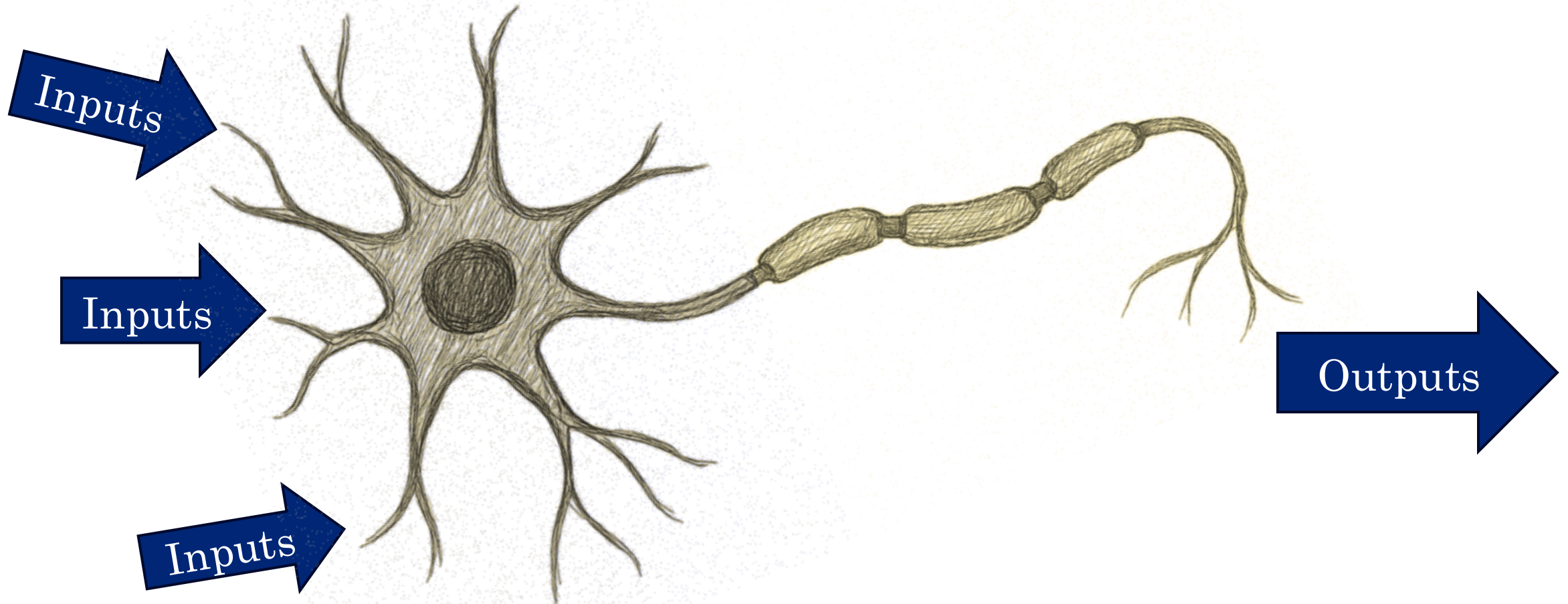
The Structure of a Biological Neuron

- Simple building block in brain: **Neuron**
- Dendrite: Collecting signals from other neurons.
- Soma (cell body): Spatial summation and processing.
- Axon: Transmitting signals to dendrites of other cells.
- Synapse: Neurons use synaptic connections to exchange information.
 - Synapses can be either **excitatory** or **inhibitory** in strength.



Starting with a Single Neuron

Simulate the **firing of a neuron** as a **weighted combination of its inputs**.

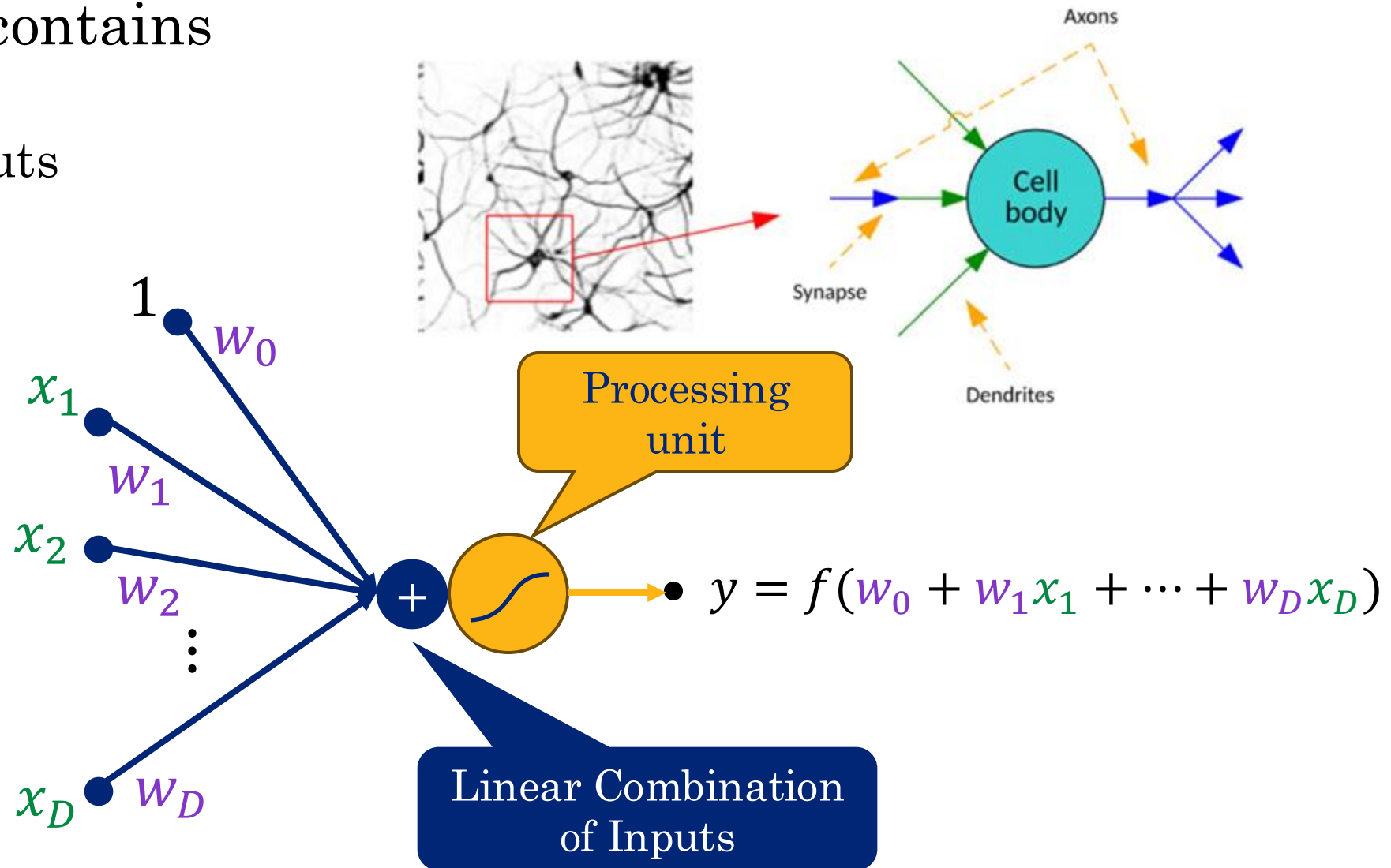


From Biological to Artificial Neurons

- Biological neuron contains

- Inputs
- Summation of inputs
- Processing unit
- Output

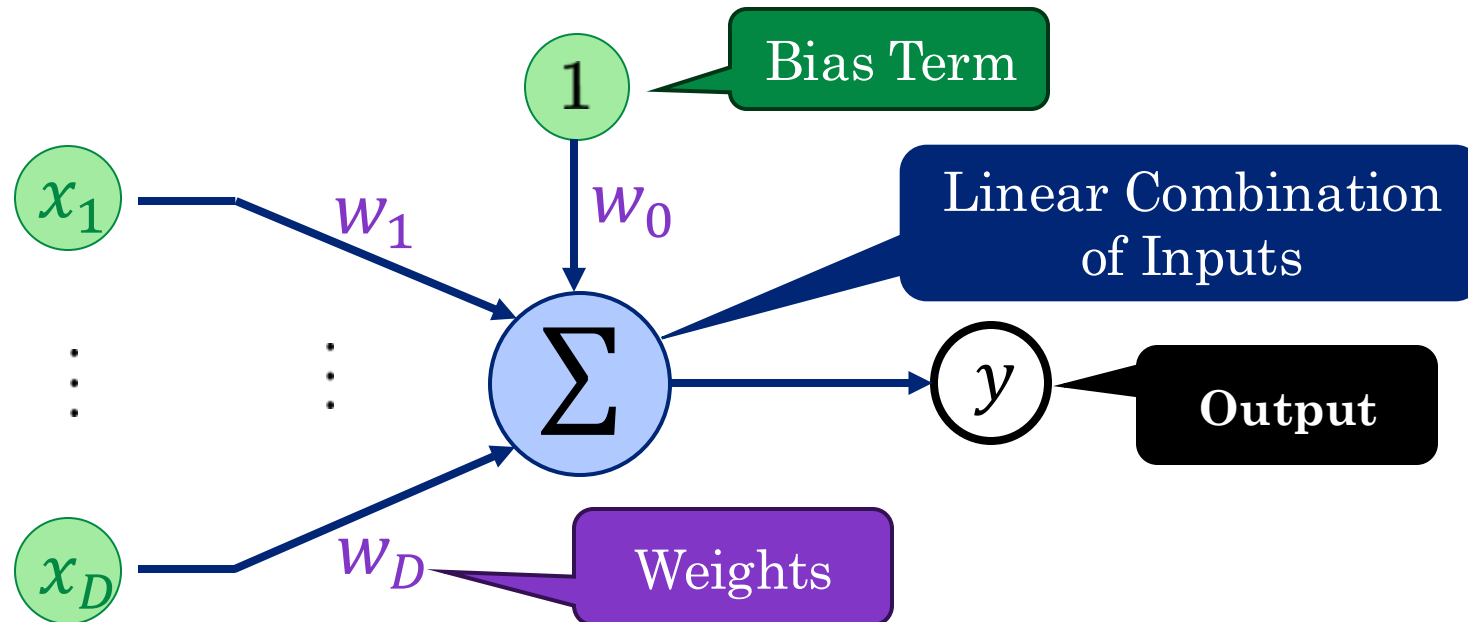
- Artificial neuron



Linear Regression in a Summation Unit

- Graphical representation
 - Inputs are x_i values entering the summation unit.
 - Connections are weighted by parameters.

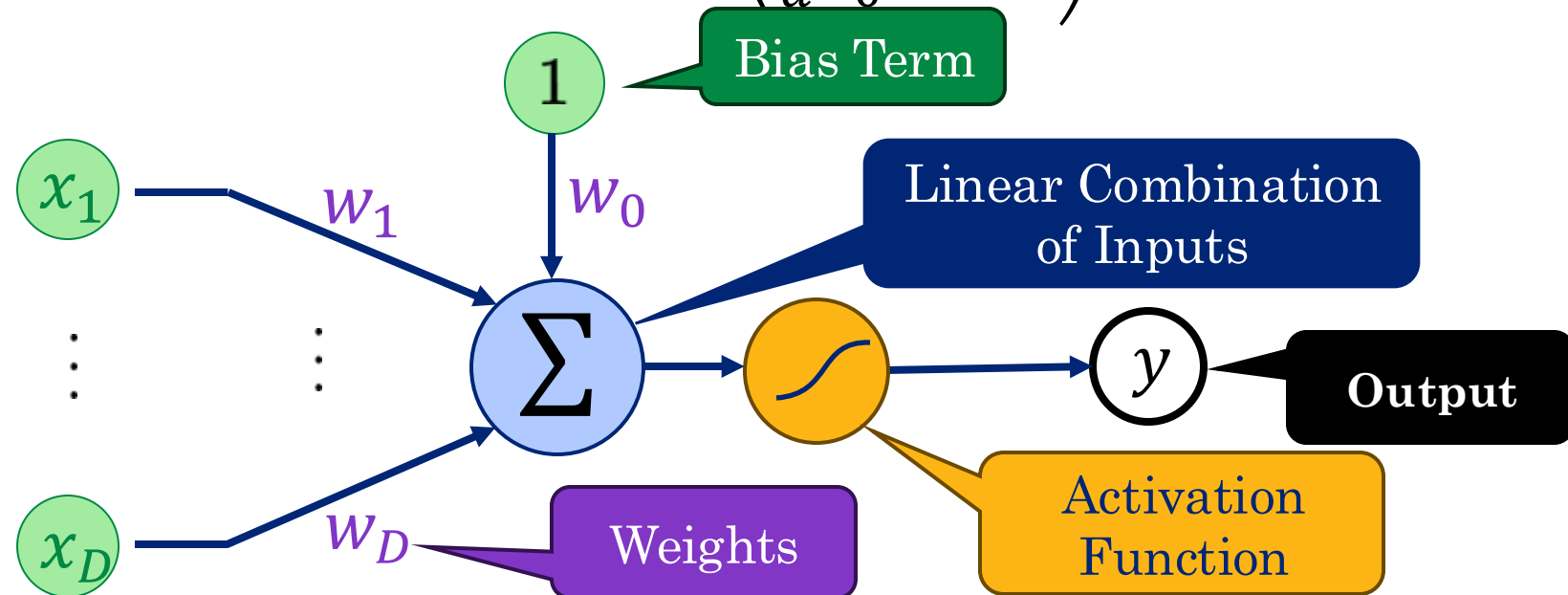
$$y = \vec{w} \cdot \vec{x} = \sum_{d=0}^D w_d x_d$$



Neuron – Activation Function

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

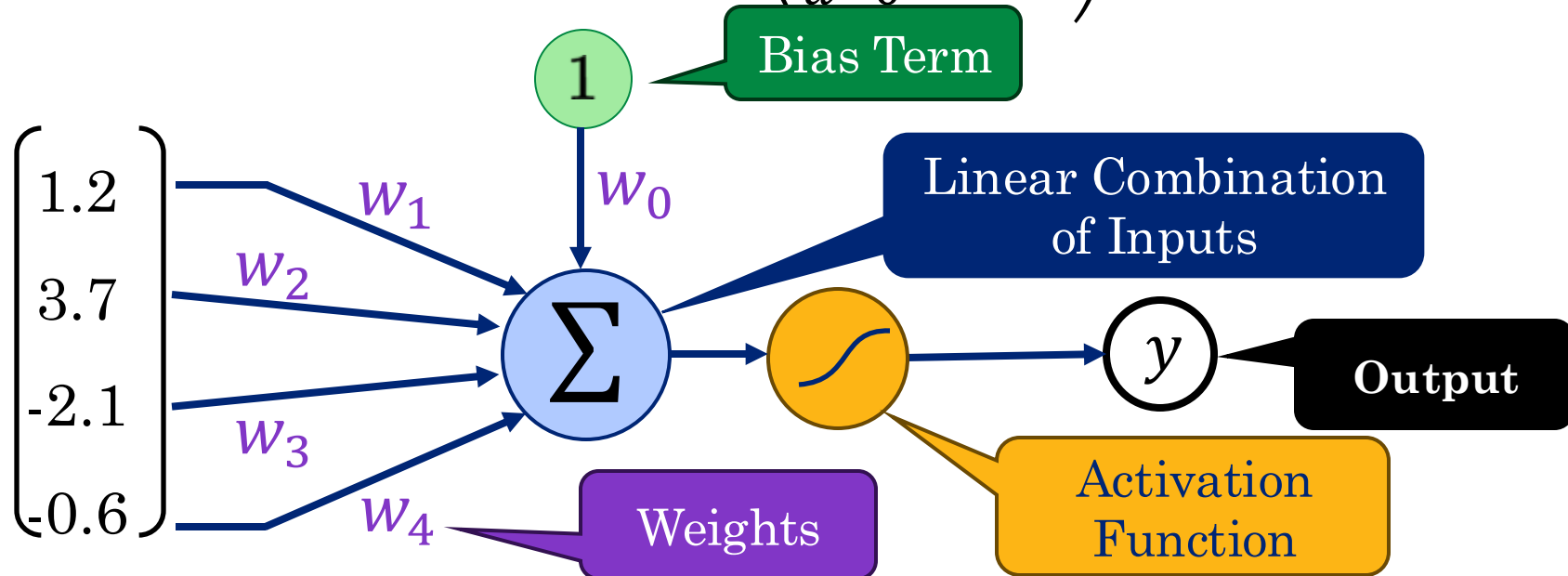
$$y = f(\vec{w} \cdot \vec{x}) = f\left(\sum_{d=0}^D w_d x_d\right)$$



A Simple Neuron

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

$$y = f(\vec{w} \cdot \vec{x}) = f\left(\sum_{d=0}^D w_d x_d\right)$$

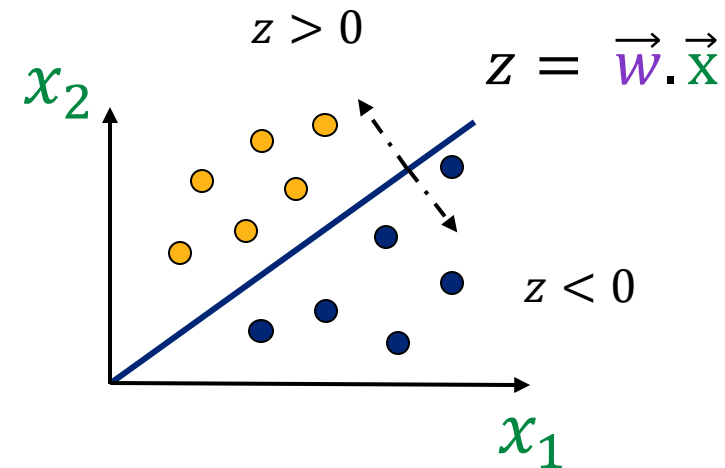
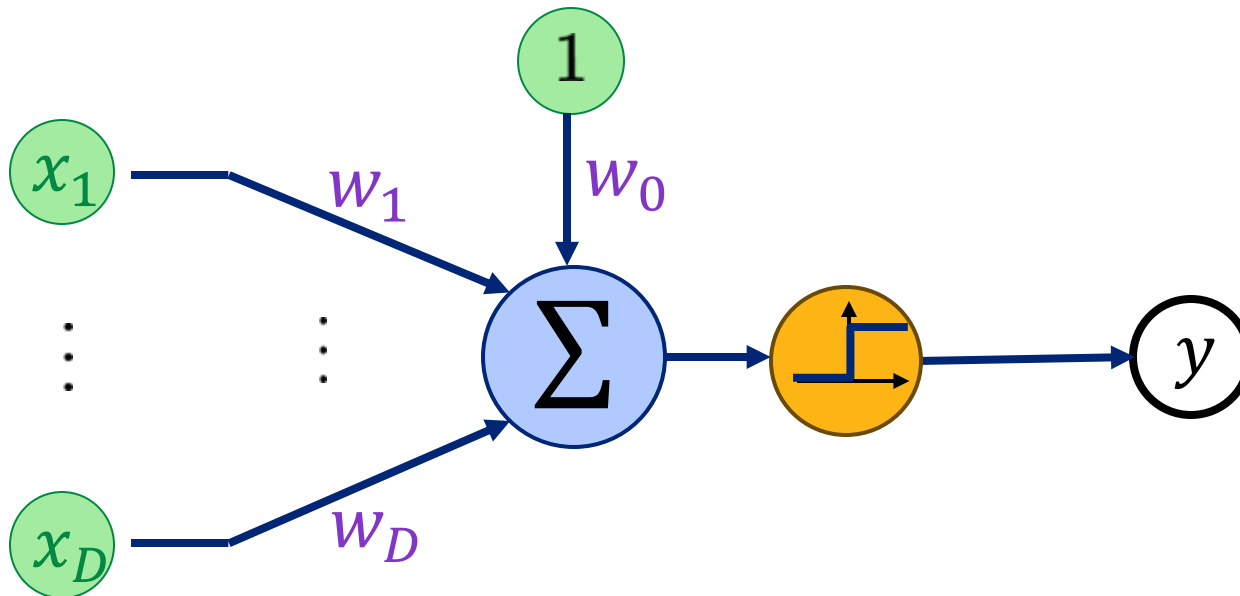


A Neuron with Step Function as Activation

- Assume the activation function to be a **step function**.

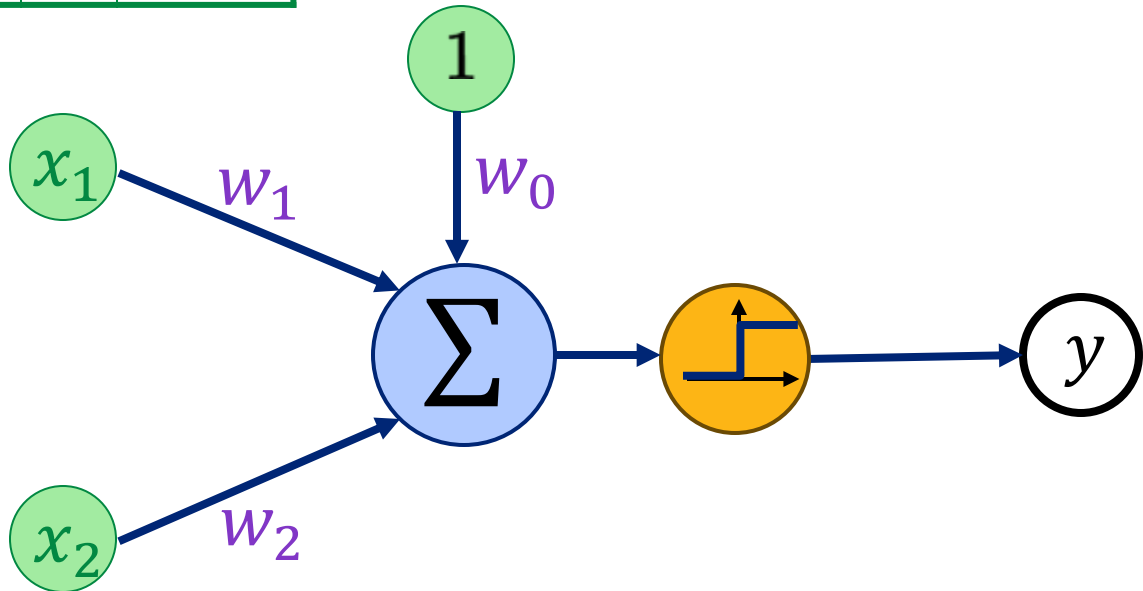
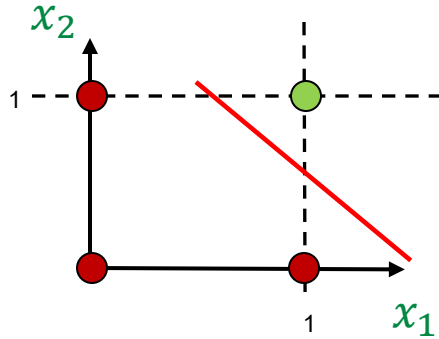
- If the weighted combination of inputs is **negative**, the output is **0** → neuron is **not activated**.
- If the weighted combination of inputs is **positive**, the output is **1** → neuron is **activated**.

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$



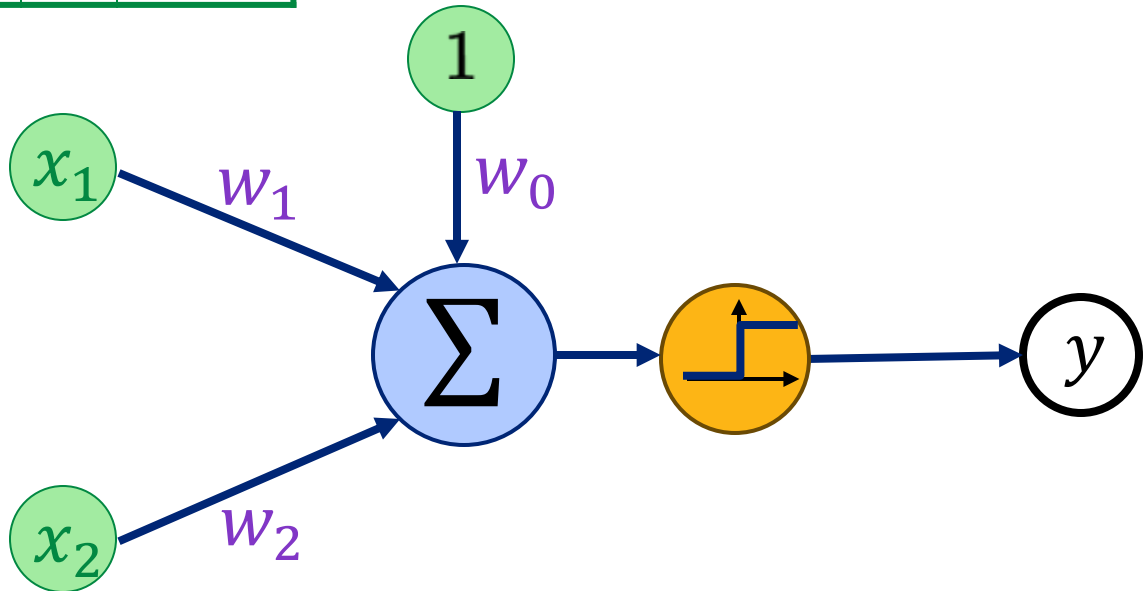
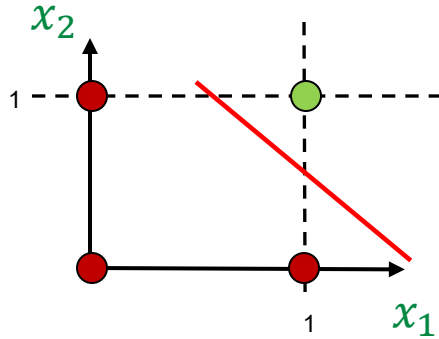
Functional Completeness - AND

x_1	x_2	y
1	0	0
1	1	1
0	0	0
0	1	0



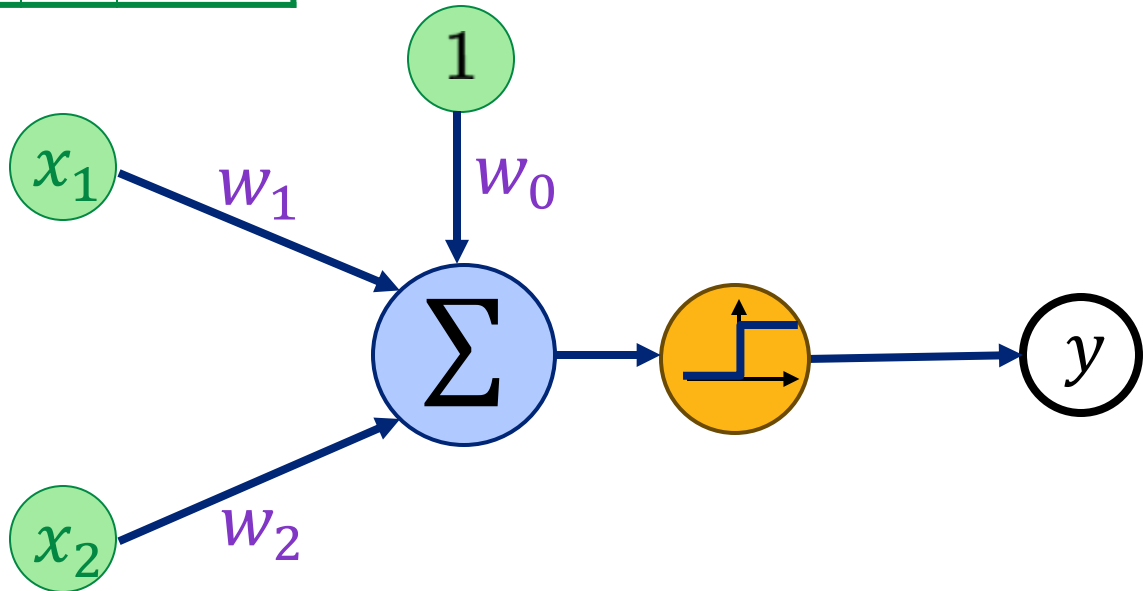
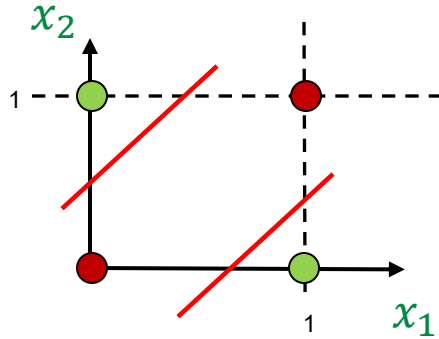
Functional Completeness - AND

x_1	x_2	y
1	0	0
1	1	1
0	0	0
0	1	0



Functional Completeness - XOR

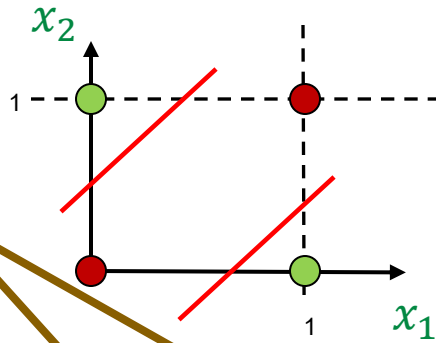
x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	1



Functional Completeness - XOR

x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	1

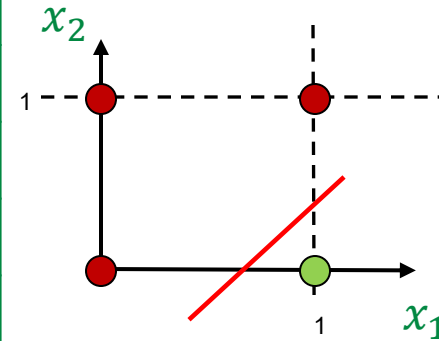
$x_1 \text{ XOR } x_2$



Non-linear Decision Boundary
(No linear separability)

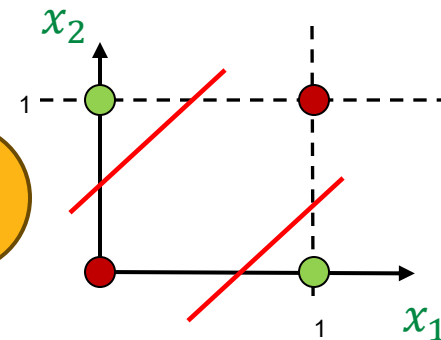
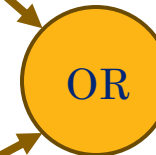
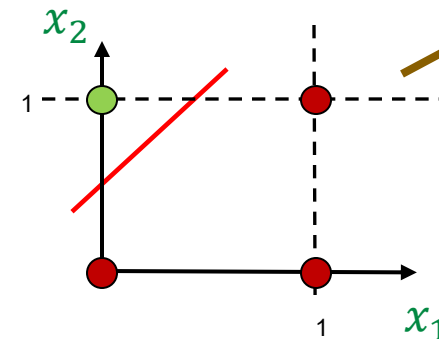
$x_1 \text{ AND } \sim x_2$

x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	0



$\sim x_1 \text{ AND } x_2$

x_1	x_2	y
1	0	0
1	1	0
0	0	0
0	1	1

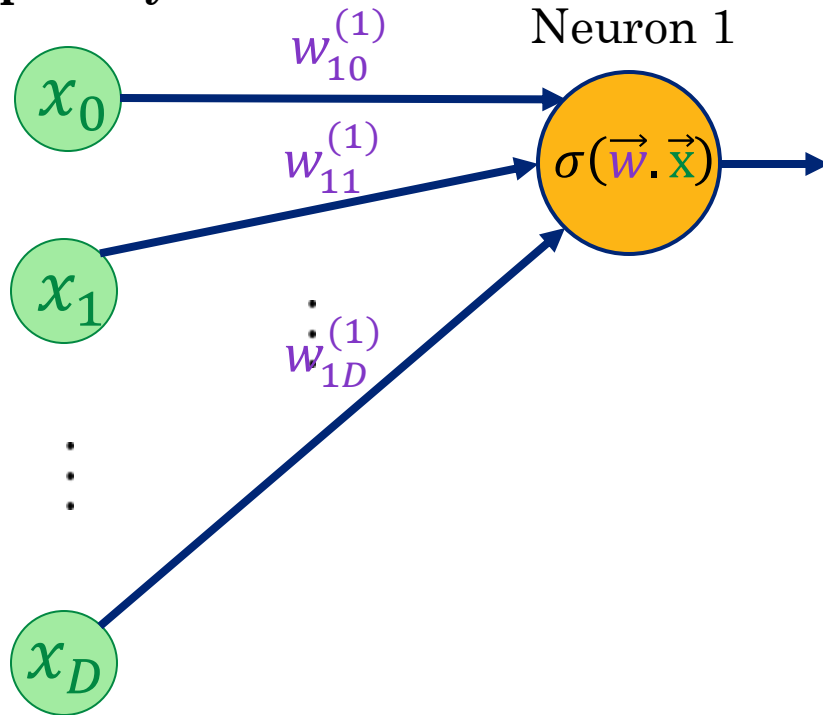


Feed-Forward Neural Network

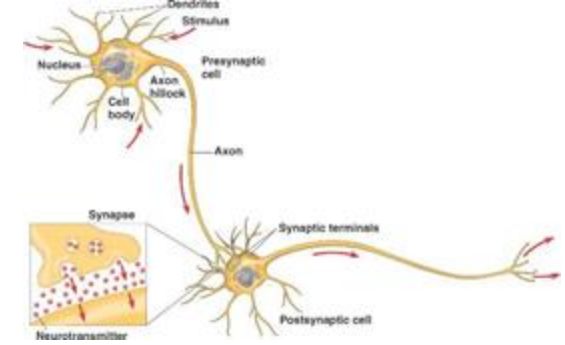
- **Limitations of Fixed Basis Functions**
- **Neural Networks**
 - **Neurons**
 - **Feed-Forward Neural Network**
- Universality Theorem
- Activation Functions
- Building & Training Neural Networks in PyTorch

Neural Network

Input layer

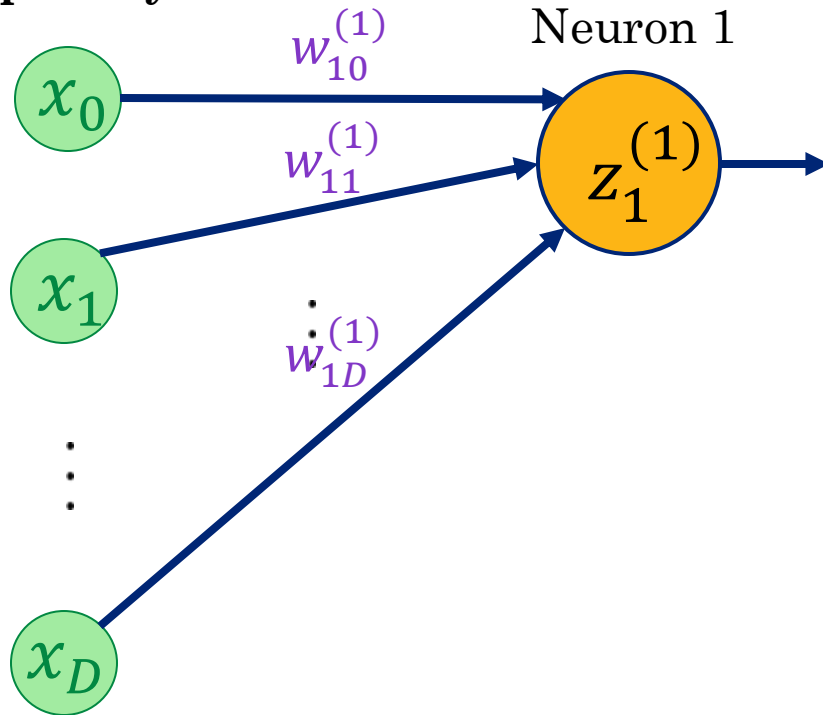


Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

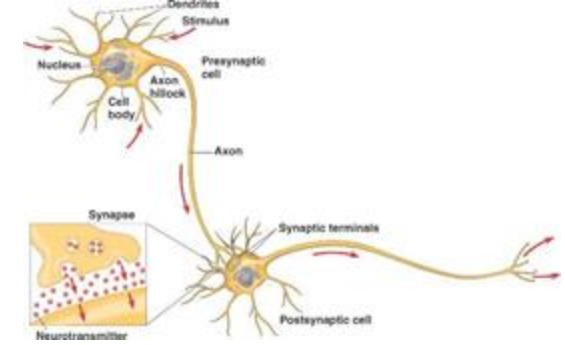


Neural Network

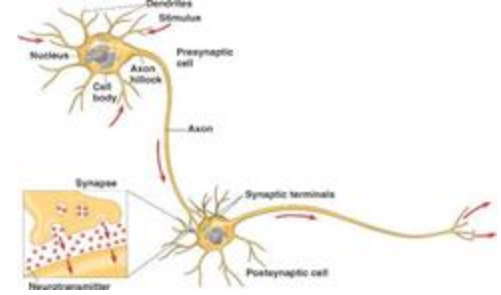
Input layer



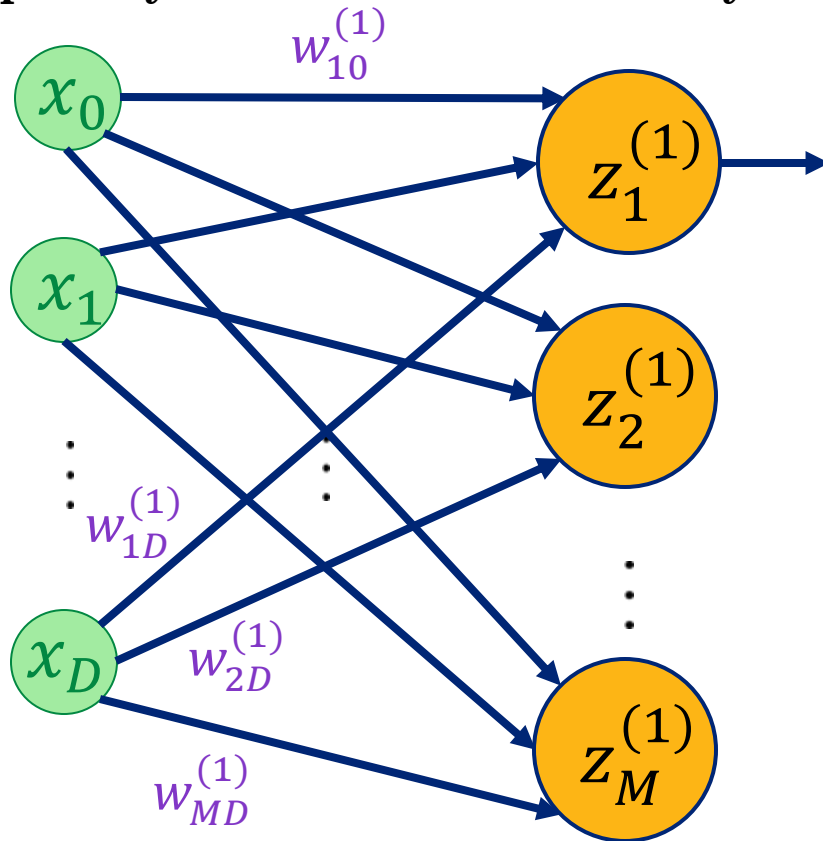
Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.



Neural Network



Input layer Hidden layer



Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

Activations (M linear combination of inputs)

$$a_j^{(1)} = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

Pre-activation or logits

Weights

Bias Term

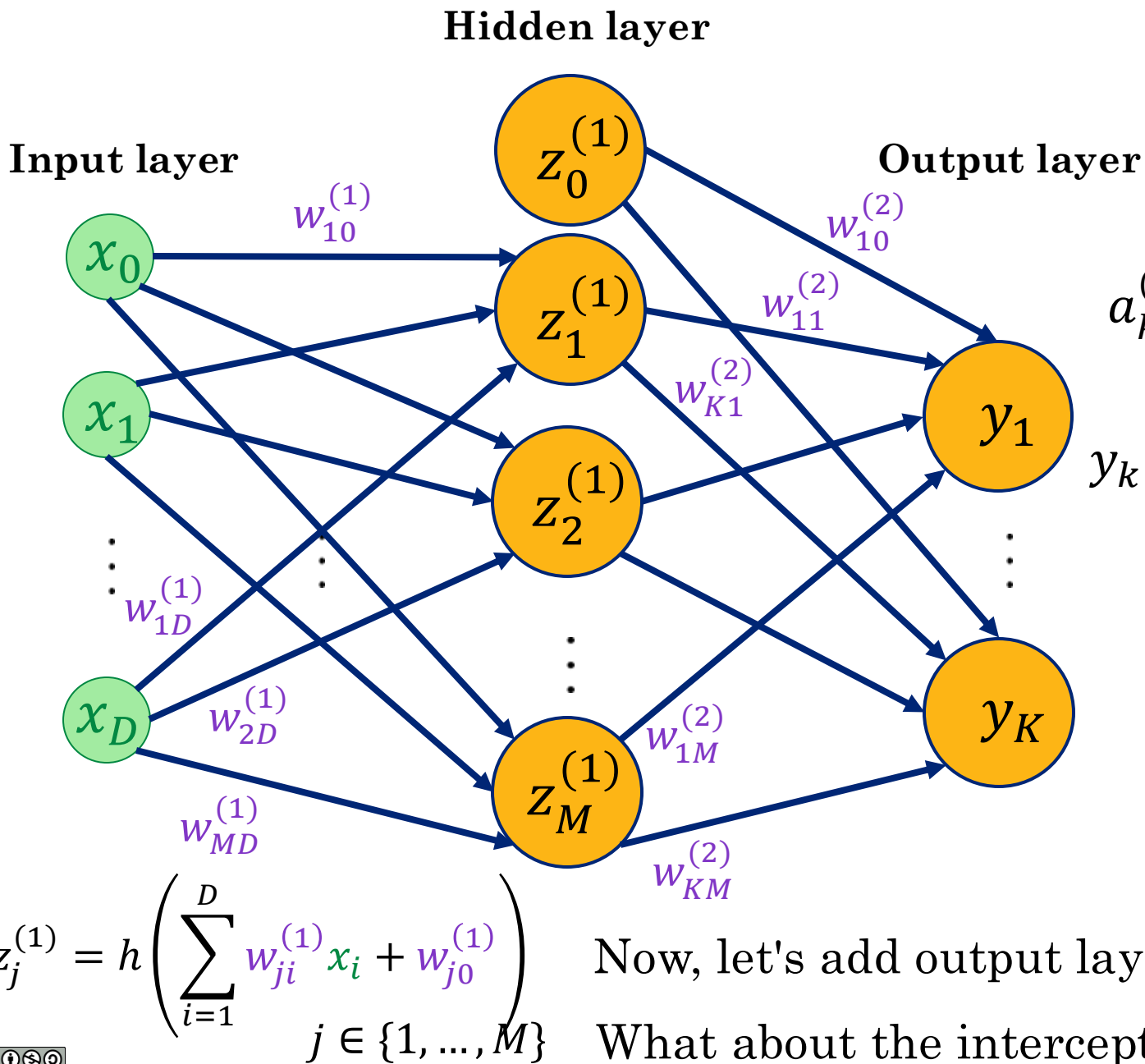
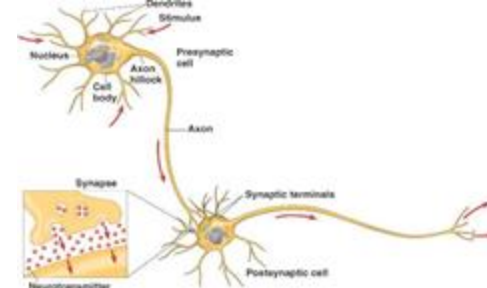
$j \in \{1, \dots, M\}$

$$z_j^{(1)} = h(a_j^{(1)})$$

Output of basis functions

Activation Function

$j \in \{1, \dots, M\}$



$$a_k^{(2)} = \sum_{j=1}^M w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)} \quad k \in \{1, \dots, K\}$$

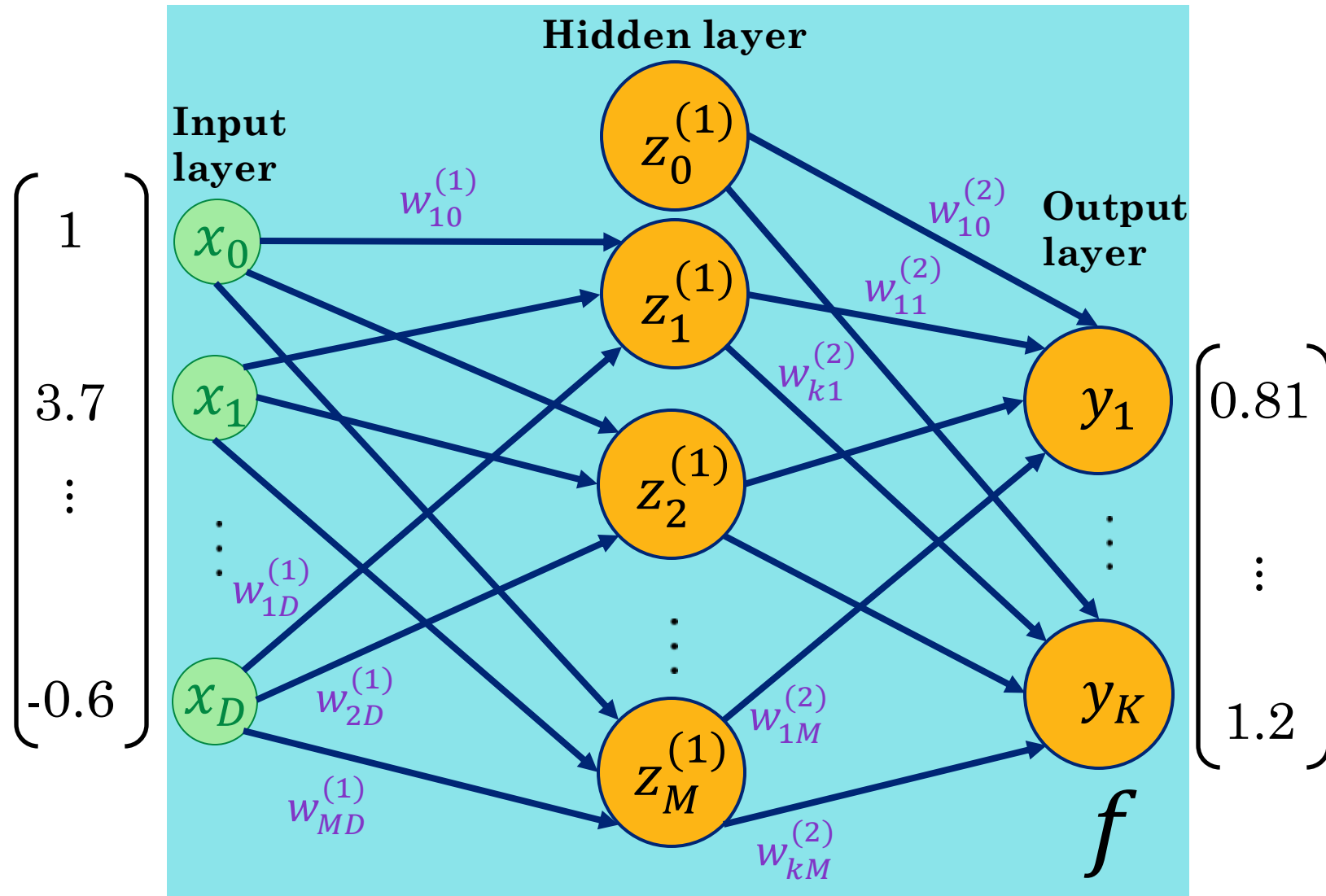
$$y_k(x, w) = f \left(a_k^{(2)} \right)$$

$$= f \left(\sum_{j=1}^M w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)} \right)$$

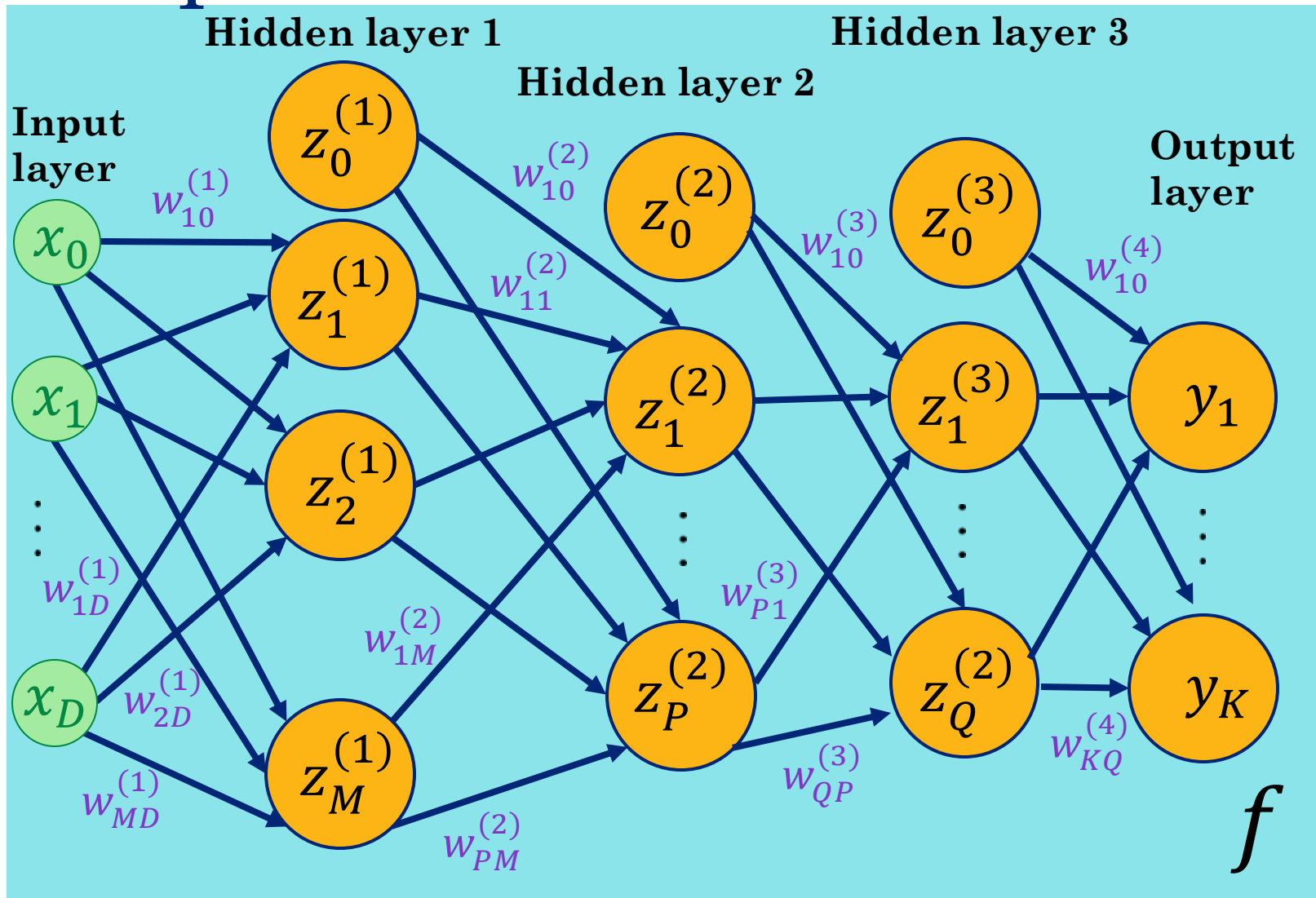
$$= f \left(\sum_{j=1}^M w_{kj}^{(2)} h \left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \right) + w_{k0}^{(2)} \right)$$

$$= f \left(W^{(2)} h(W^{(1)} \mathbf{x}) \right)$$

Neural Network



Deep Neural Network



$$l \in \{1, \dots, L\}$$

$$z^{(l)} = h^{(l)}(W^{(l)} z^{(l-1)})$$

Activation in
layer l

Weights in
layer l

$$z^{(0)} = \mathbf{x}$$

$$z^{(L)} = \mathbf{y}$$

Universality Theorem

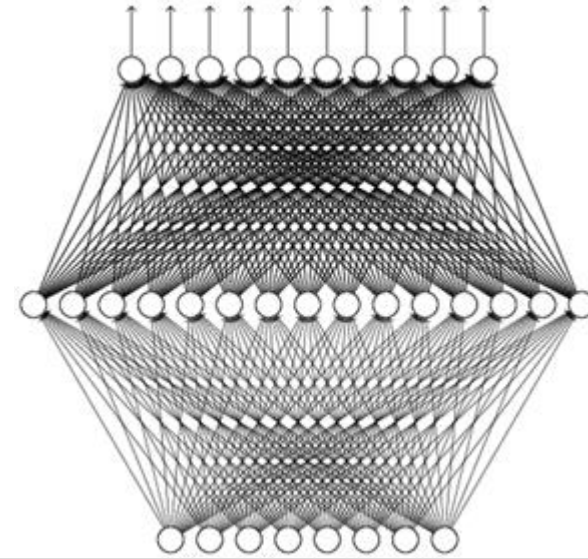
- Limitations of Fixed Basis Functions
- Neural Networks
 - Neurons
 - Feed-Forward Neural Network
- **Universality Theorem**
- Activation Functions
- Building & Training Neural Networks in PyTorch

Universal Approximation Theorem

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^K$$

(Cybenko 1989, Funahashi, 1989)

Any function defined over a continuous subset of $\mathbb{R}^D$ can be approximated to arbitrary accuracy by a neural network with one hidden layer.



Visual:

<http://neuralnetworksanddeeplearning.com/chap4.html>

Neural networks are universal function approximators.

Why “Deep” neural network not “Fat” neural network?

Universal Approximation Theorem

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^K$$

(Cybenko 1989, Funahashi, 1989)

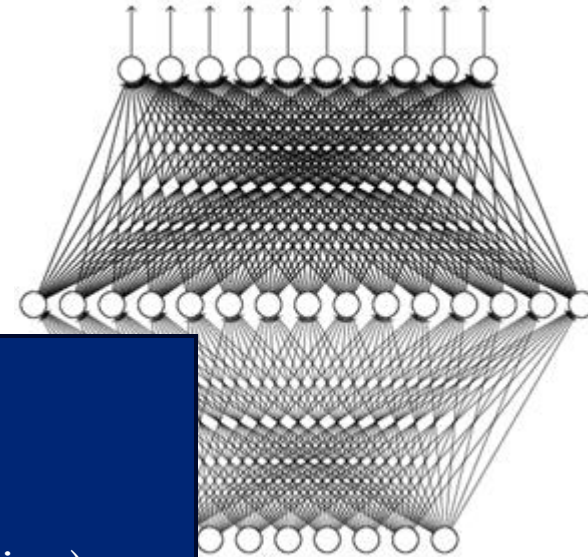
Any function defined over a

compact
subset of
 $\mathbb{R}^D$
can be
approximated
by a
finite
number of
hidden units

Doesn't tell us how to find the right weights (Training)

How big is the hidden layer?

What activation function to use



Visual:

neuralnetworksanddeeplearning.com/chap4.html

approximators.

” **neural network?**

Universal Approximation Theorem

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^K$$

(Cybenko 1989, Funahashi, 1989)

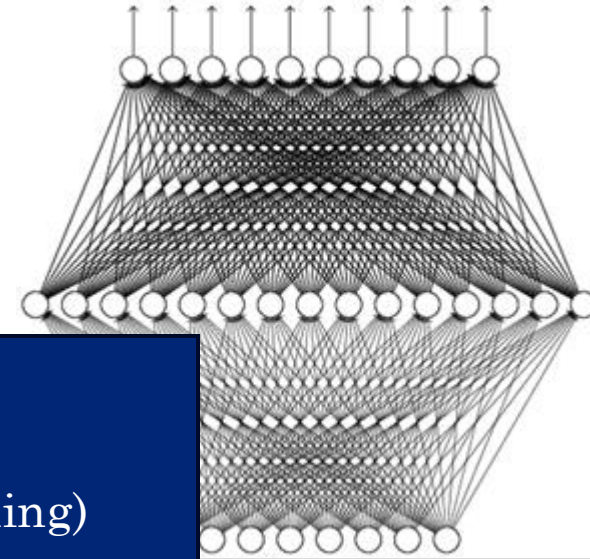
Any function defined over a

compact
subset of
 $\mathbb{R}^D$
can be
approximated
by a
neural network

Doesn't tell us how to find the right weights (Training)
Gradient Descent

How big is the hidden layer?
Exponential in D.

What activation function to use
ReLU



Visual:

neuralnetworksanddeeplearning.com/chap4.html

approximators.

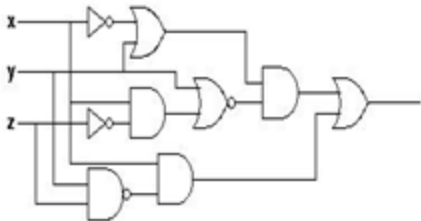
” **neural network?**

Why Deep? Analogy

Logic circuits

- Logic circuits consists of **gates**.
- **Two layers of logic gates** can represent **any Boolean function**.
- Using multiple layers of logic gates to build some functions can be exponentially simpler

→ less gates needed



Neural network

- Neural network consists of **neurons**.
- **A hidden layer network** can represent **any continuous function**.
- Using multiple layers of neurons to represent some functions can be exponentially simpler



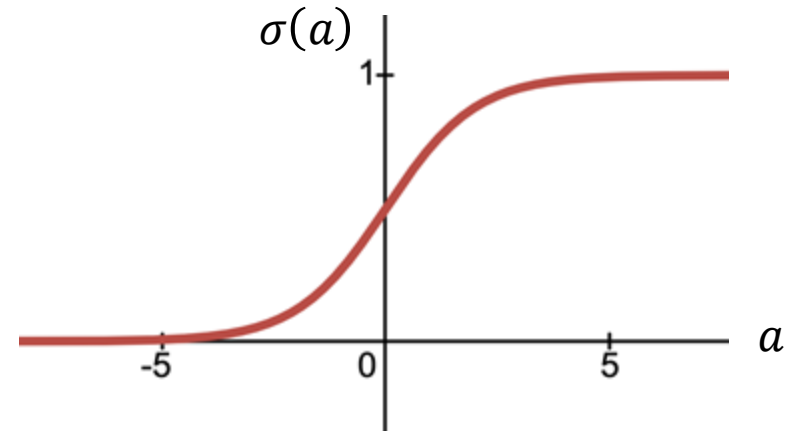
Activation Functions

- Limitations of Fixed Basis Functions
- Neural Networks
 - Neurons
 - Feed-Forward Neural Network
- Universality Theorem
- Activation Functions
- Building & Training Neural Networks in PyTorch

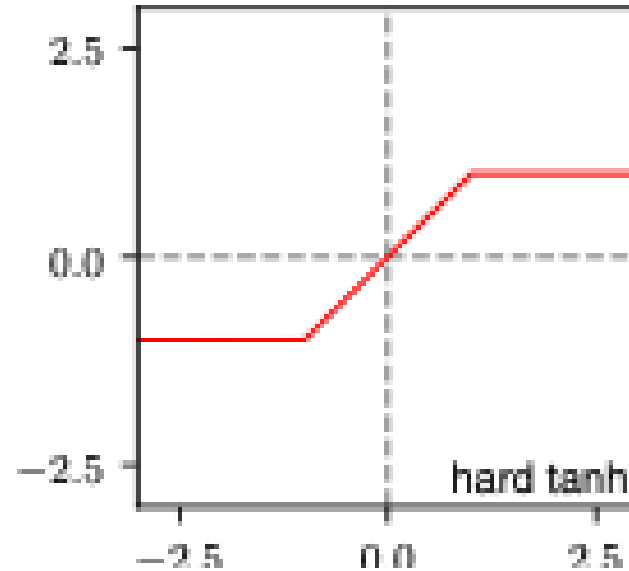
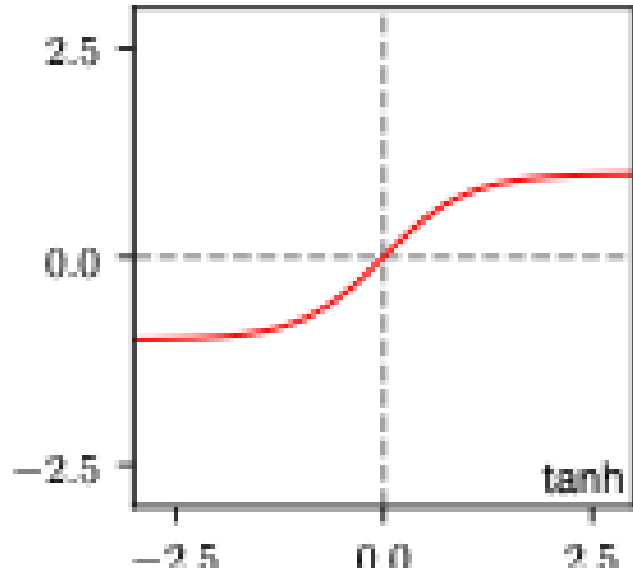
Activation Functions

- The simplest option for a hidden unit activation function is the identity function.
 - But it is redundant to have a hidden layer if all activations are identity.
- Simple non-linear differentiable activation function is sigmoid.

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



Other Activation Functions



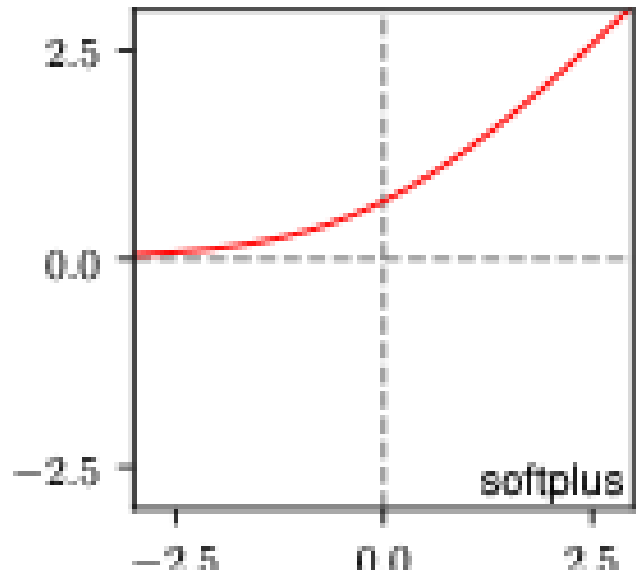
$$\tanh(a) = 2\sigma(2a) - 1 \quad h(a) = \max(-1, \min(1, a))$$

$$= \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

One issue with tanh and sigmoid family action function is that gradients go to 0 when magnitude of activations are large.

We need an activation that has non-zero gradient at least when activations are large positive values.

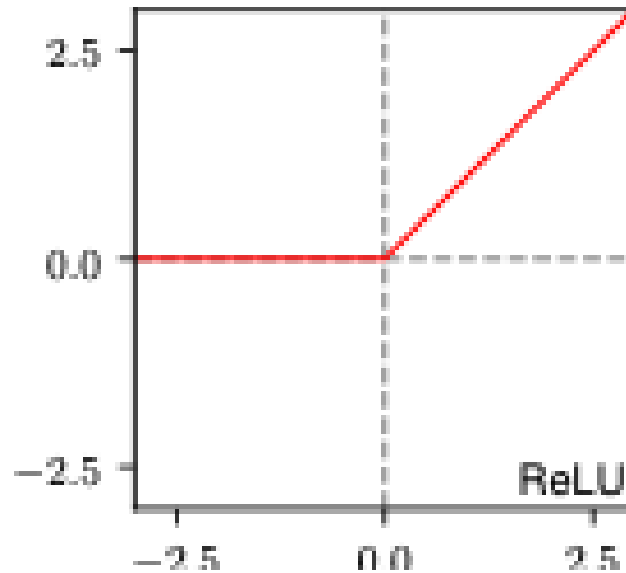
Other Activation Functions



$$h(a) = \ln(1 + \exp(a))$$

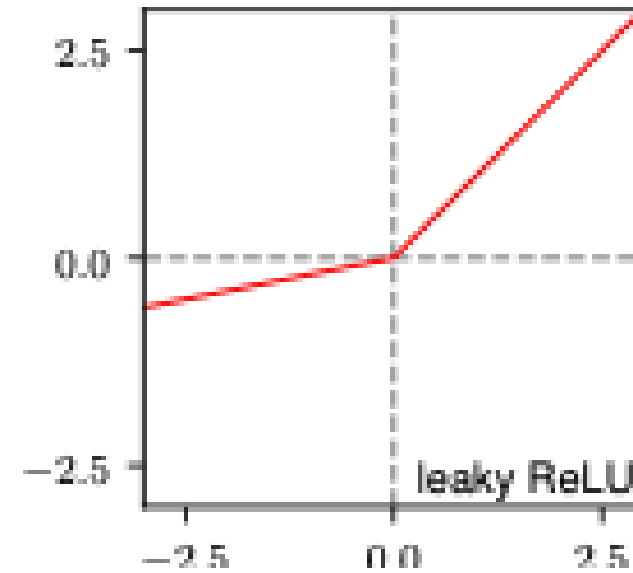
When $a \gg 1$, $h(a) \simeq a$

Also called soft ReLU



$$h(a) = \max(0, a)$$

Neurons with negative activations will receive "no error signal"



$$h(a) = \max(0, a) + \alpha \min(0, a)$$

Why hidden layers ?

In machine learning you are given a vector $\mathbf{x}$ of d features. You often expand it into say d^2 features and get better performance.

This can be done in three ways:

Why hidden layers ?

In machine learning you are given a vector $\mathbf{x}$ of d features. You often expand it into say d^2 features and get better performance.

This can be done in three ways:

1. Problem specific feature engineering

Why hidden layers ?

In machine learning you are given a vector $\mathbf{x}$ of d features. You often expand it into say d^2 features and get better performance.

This can be done in three ways:

1. Problem specific feature engineering
2. Embed in very general space and use Kernels.

Why hidden layers ?

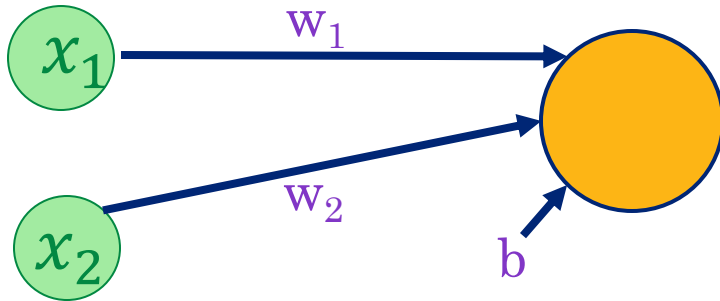
In machine learning you are given a vector $\mathbf{x}$ of d features. You often expand it into say d^2 features and get better performance.

This can be done in three ways:

1. Problem specific feature engineering
2. Embed in very general space and use Kernels.
3. Learn the features from data.

You can do this if the model has end-to-end differentiation.

let's work on an example



Let's start with a linear model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\mathbf{w} = \{w_1, w_2, b\}$

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Training=Learning= Fitting = Optimizing parameters $\mathbf{w}$.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$$

Write down the loss $J(\mathbf{w})$

Do gradient descent or SGD or Mini-Batch SGD on $J(\mathbf{w})$ to find the best parameters $\mathbf{w}^*$.

Goal: Find what that minimizes $J(\mathbf{w})$.

Loss= $J(\boldsymbol{\theta})$ = squared loss...

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a linear model:

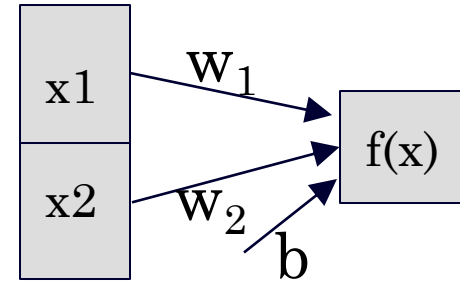
$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\mathbf{w} = \{w_1, w_2, b\}$

$$\text{Loss} = J(\boldsymbol{\theta}) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
weights $\boldsymbol{\theta} = \{w_1, w_2, b\}$

Loss $J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$

Exercise: Compute

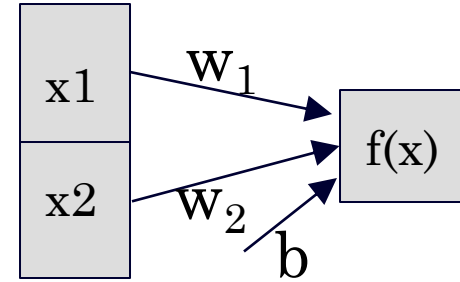
1. predictions $f(\mathbf{x})$ and
2. loss for $J(\boldsymbol{\theta})$

For $\boldsymbol{\theta} = \{w_1 = \mathbf{0}, w_2 = \mathbf{1}, b = \mathbf{1}\}$

using the data set shown in the table

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
weights $\boldsymbol{\theta} = \{w_1, w_2, b\}$

$$\text{Loss} = J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Answer:

$$f(\mathbf{x}) = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \end{bmatrix}$$

$$\begin{aligned} \text{Loss } J(\boldsymbol{\theta}) &= \frac{1}{4} * ((0-1)^2 + (1-2)^2 \\ &\quad + (1-1)^2 + (0-2)^2) \\ &= \frac{1}{4} * (1+1+0+4) = \mathbf{6/4} \end{aligned}$$

Exercise: Compute

1. predictions $f(\mathbf{x})$ and
2. loss for $J(\boldsymbol{\theta})$

For $\boldsymbol{\theta} = \{w_1 = \mathbf{0}, w_2 = \mathbf{1}, b = \mathbf{1}\}$

using the data set shown in the table

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\{w_1, w_2, b\}$

$$J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Exercise:

1. Write the loss function for this dataset
2. Compute the gradient of the loss function.
3. For $\theta = \{w_1 = \mathbf{0}, w_2 = \mathbf{1}, b = \mathbf{1}\}$ do a gradient step using step size $\eta = 0.5$. Is the point $\theta = \{\mathbf{0}, \mathbf{1}, \mathbf{1}\}$ a local minimum ?

$$\text{Loss} = J(\theta) = \frac{1}{4} ..$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\theta = \{w1, w2, b\}$

$$J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Exercise:

1. Write the loss function for this dataset
2. Compute the gradient of the loss function.
3. For $\theta = \{w1=\mathbf{0}, w2=\mathbf{1}, b=\mathbf{1}\}$ do a gradient step using step size $\eta=0.5$. Is the point $\theta = \{\mathbf{0}, \mathbf{1}, \mathbf{1}\}$ a local minimum ?

$$\text{Loss} = J(\theta) = \frac{1}{4} ((0 - ((0,0) \cdot (w1, w2) + b))^2 + \dots$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

1. Write the loss function

$$\begin{aligned} \text{Loss} = J(\theta) = \frac{1}{4} (& (0 - (\\ & (0,0) * (w1,w2) + b))^2 \\ & + (1 - (0,1) * (w1,w2) + b))^2 + \\ & + (1 - (1,0) * (w1,w2) + b))^2 + \\ & + (0 - (1,1) * (w1,w2) + b))^2. \end{aligned}$$

Compute the gradient of the loss
and move the model !

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

1. Write the loss function

$$\begin{aligned} \text{Loss} = J(\theta) = \frac{1}{4} & \left((0 - (0,0) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right. \\ & + (1 - (0,1) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & + (1 - (1,0) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & \left. + (0 - (1,1) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right). \end{aligned}$$

$$\begin{aligned} = \frac{1}{4} & \left((0 - b)^2 \right. \\ & + (1 - \mathbf{w}_2 - b)^2 + \\ & + (1 - \mathbf{w}_1 - b)^2 + \\ & \left. + (0 - \mathbf{w}_1 - \mathbf{w}_2 - b)^2 \right) \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth
1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss
function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) &= \frac{1}{4} [(0 - (0w_1 + 0w_2 + b))^2 \\ &\quad + (1 - (0w_1 + 1w_2 + b))^2 + \\ &\quad (0 - (0w_1 + 1w_2 + b))^2 + (1 - (1w_1 + 1w_2 + b))^2] \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth
1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss
function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) &= \\ &= \frac{1}{4} [(0 - b))^2 \\ &\quad + (1 - w_2 - b))^2 + \\ &\quad (1 - w_1 - b))^2 + (0 - w_1 - w_2 - b))^2] \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth
1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss
function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) = & \frac{1}{4} [b^2 \\ & + (1 - w_2 - b))^2 + \\ & (1 - w_1 - b))^2 + (0 - w_1 - w_2 - b))^2] \end{aligned}$$

Now find the gradient of the loss

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.

The unknown parameters are $\mathbf{w} = \{w_1, w_2, b\}$
 $d=3$, $n=4$.

The gradient is a vector.
first coordinate is ...?

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The gradient is a vector.

first coordinate is dJ / dw_1

Second is dJ / dw_2

Third is dJ / db

Therefore first coordinate of gradient is computed:

$$\frac{d}{dw_1} \frac{1}{4} [b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (-w_1 - w_2 - b)^2]$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The gradient is a vector.

first coordinate is dJ / dw_1

Second is dJ / dw_2

Third is dJ / db

Therefore first coordinate of gradient is computed:

$$\frac{d}{dw_1} \frac{1}{4} [b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (-w_1 - w_2 - b)^2]$$

$$\frac{1}{4} \left[\frac{d}{dw_1} b^2 + \frac{d}{dw_1} (1 - w_2 - b)^2 + \frac{d}{dw_1} (1 - w_1 - b)^2 + \frac{d}{dw_1} (-w_1 - w_2 - b)^2 \right]$$

$$\frac{1}{4} [0 + 0 + -2(1 - w_1 - b) + -2(-w_1 - w_2 - b)] = \frac{1}{2} (2w_1 + w_2 + 2b - 1)$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Gradient at (0,1,1)=

Let's start with a depth 1 model:
 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
 The unknown parameters are $\theta = \{w_1, w_2, b\}$
 $d=3$, $n=4$.

$$\text{Loss} = J(\theta) = \frac{1}{4} \left((0 - b)^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2 \right)$$

The gradient is a vector.
 first coordinate is dJ / dw_1
 Second is $dJ / dw_2 = 1/2$
 Third is dJ / db

$2w_1 + w_2 + 2b - 1$
$w_1 + 2w_2 + 2b - 1$
$2w_1 + 2w_2 + 4b - 2$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Gradient at (0,1,1)=

1
3/2
2

Let's start with a depth 1 model:
 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
 The unknown parameters are $\theta = \{w_1, w_2, b\}$
 $d=3$, $n=4$.

$$\text{Loss} = J(\theta) = \frac{1}{4} \left((0 - b)^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2 \right)$$

The gradient is a vector.
 first coordinate is dJ / dw_1
 Second is $dJ / dw_2 = 1/2$
 Third is dJ / db

$2w_1 + w_2 + 2b - 1$
$w_1 + 2w_2 + 2b - 1$
$2w_1 + 2w_2 + 4b - 2$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

After doing gradient descent for many steps you can train this linear model and find the optimal weights: $w^*=[0,0]$, $b^*=1/2$

If you put input $x=[1,0]$ to this optimal model what is the output?

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

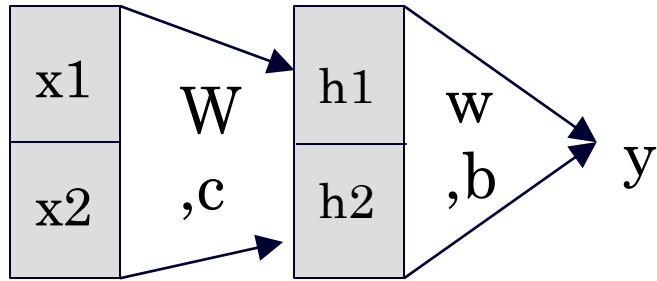
After doing gradient descent for many steps you can train this linear model and find the optimal weights: $w^*=[0,0]$, $b^*=1/2$

If you put input $x=[1,0]$ to this optimal model what is the output?

Model predicts $\frac{1}{2}$ for any input. Didn't learn the XOR function.

What about a 1-hidden layer deep linear model?

1 hidden layer- Deep linear model

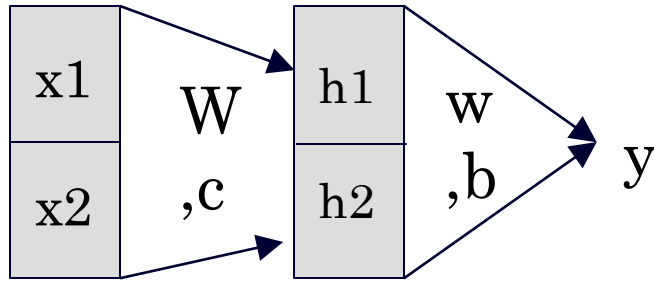


$$\mathbf{h} = W^T \mathbf{x} + \mathbf{c}$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

How many model parameters does this network have?

Can this network learn the XOR function?

1 hidden layer- Deep linear model



Model weights: $4+2$, $2+1$

Total: 9 parameters

$$\mathbf{h} = W^T \mathbf{x} + \mathbf{c}$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

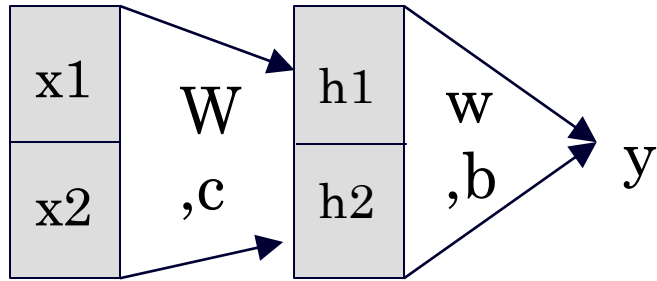
How many model parameters does this network have?

Can this network learn the XOR function?

See example from Ch6 GoodFellow et al. Deep Learning Book:

<https://www.deeplearningbook.org/contents/mlp.html>

1 hidden layer- linear



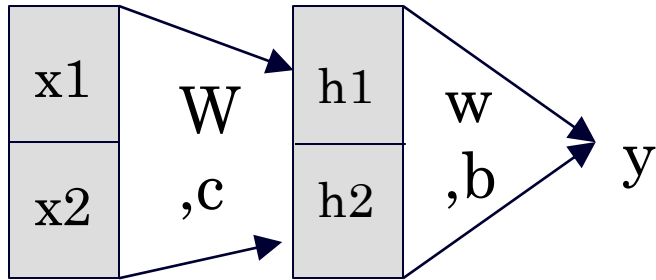
$$\mathbf{h} = W^T \mathbf{x} + \mathbf{c}$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

$$\text{So } y = \mathbf{w}^T (W^T \mathbf{x} + \mathbf{c}) + b$$
$$y = \mathbf{w}^T W^T \mathbf{x} + \mathbf{w}^T \mathbf{c} + b$$
$$y = \mathbf{w}'^T \mathbf{x} + b'$$

See example from Ch6 GoodFellow et al. Deep Learning Book:

<https://www.deeplearningbook.org/contents/mlp.html>

1 hidden layer- linear



A deep linear model with any number of layers is just a linear model end-to-end.

So it will always produce the $\frac{1}{2}$ constant prediction for any input $\mathbf{x}$

$$\mathbf{h} = W^T \mathbf{x} + \mathbf{c}$$

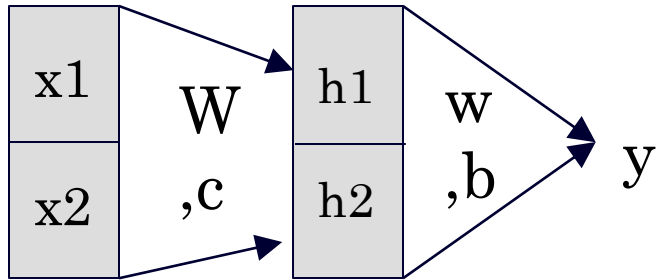
$$y = \mathbf{w}^T \mathbf{h} + b$$

$$\text{So } y = \mathbf{w}^T (W^T \mathbf{x} + \mathbf{c}) + b$$

$$y = \mathbf{w}^T W^T \mathbf{x} + \mathbf{w}^T \mathbf{c} + b$$

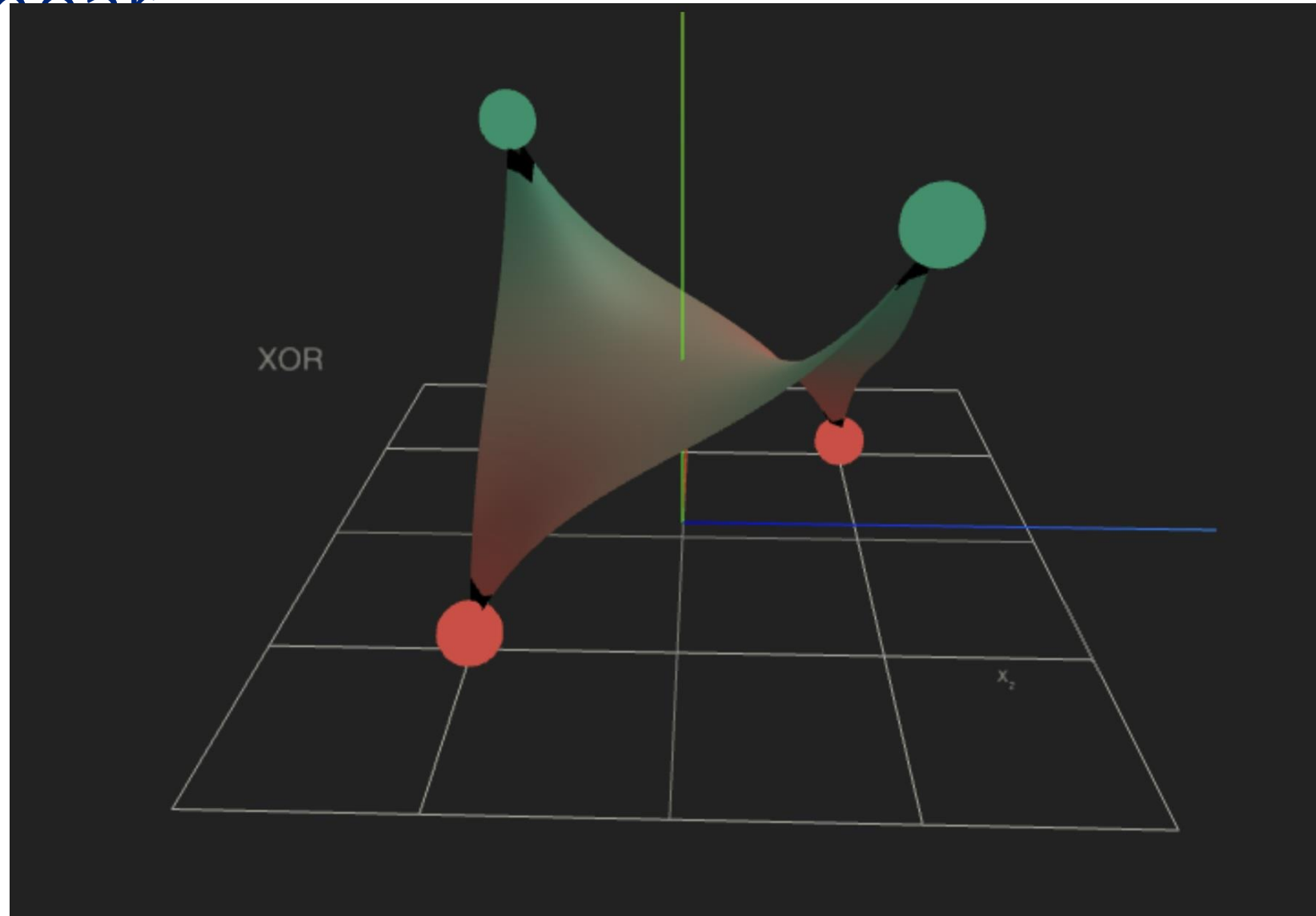
$$y = \mathbf{w}'^T \mathbf{x} + b'$$

1 hidden layer- linear

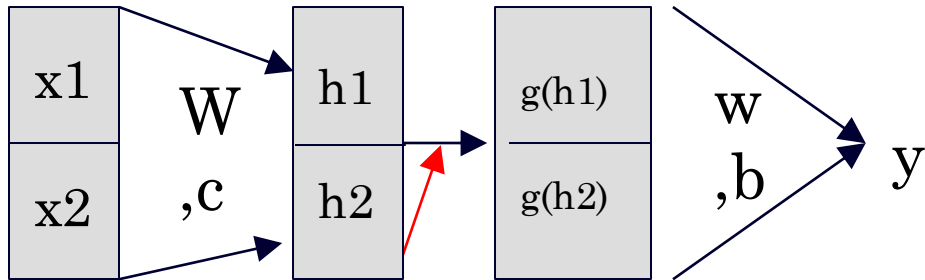


A deep linear model with any number of layers is just a linear model end-to-end.

So it will always produce the $\frac{1}{2}$ constant prediction for any input $\mathbf{x}$



1 hidden layer- ReLu Activation

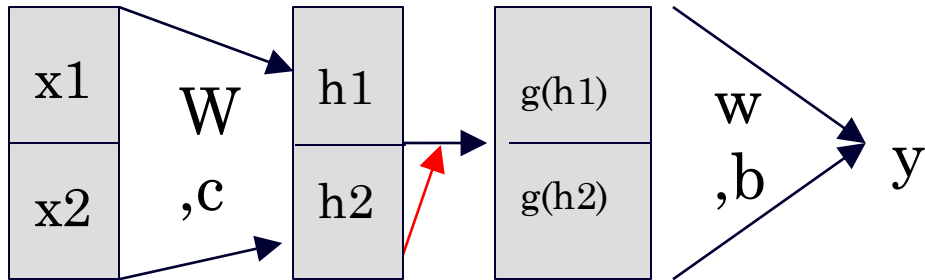


$$\mathbf{h} = g(W^T \mathbf{x} + \mathbf{c})$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

Try $W = [1, 1; 1, 1]$ $c = [0, -1]^T$
 $\mathbf{w} = [1, -2]^T$ and $b = 0$
Say $\mathbf{x} = [0, 0]$

$g(z) = \max(z, 0)$
ReLU Activation function

1 hidden layer- ReLu Activation



$g(z) = \max(z, 0)$
ReLU Activation function

$$\mathbf{h} = g(W^T \mathbf{x} + \mathbf{c})$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

Try $W = [1, 1; 1, 1]$ $c = [0, -1]^T$

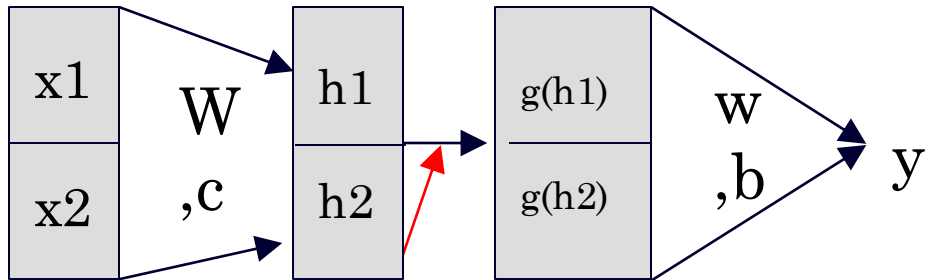
$\mathbf{w} = [1, -2]^T$ and $b = 0$

Say $\mathbf{x} = [0, 0]$

$\mathbf{h}(0, 0) = g([0, -1]^T) = [0, 0]^T$

$y = 0$

1 hidden layer- ReLu



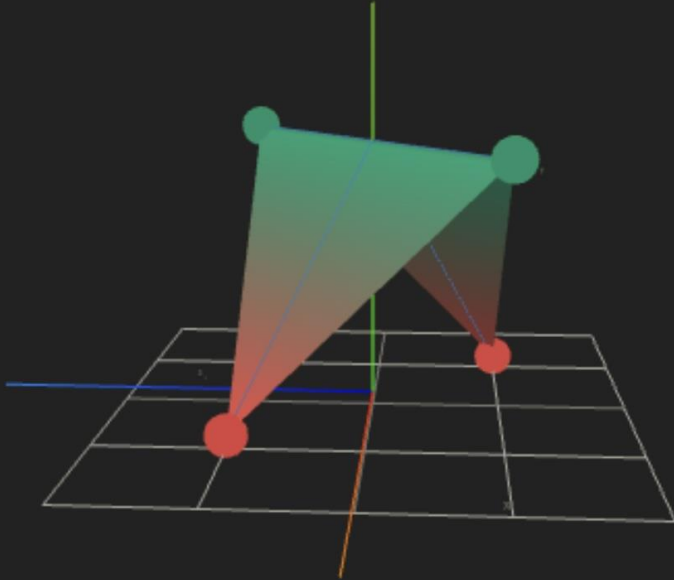
$g(z) = \max(z, 0)$
ReLU Activation function

$$\mathbf{h} = g(W^T \mathbf{x} + \mathbf{c})$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

Try $W = [1, 1; 1, 1]$ $c = [0, -1]^T$
 $\mathbf{w} = [1, -2]^T$ and $b = 0$
Say $\mathbf{x} = [0, 1]$

$$\mathbf{h}(0, 1) = g([1, 0]^T) = [1, 0]^T$$
$$f(0, 1) = 1$$

1 hidden layer- ReLu



$$\mathbf{h} = g(\mathbf{W}^T \mathbf{x} + \mathbf{c})$$

$$y = \mathbf{w}^T \mathbf{h} + b$$

$$\text{Try } \mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{c} = \begin{bmatrix} 0 & -1 \end{bmatrix}^T$$

$$\mathbf{w} = \begin{bmatrix} 1 & -2 \end{bmatrix}^T \text{ and } b = 0$$

$$\mathbf{h}(0,1) = g(\begin{bmatrix} 1 & 0 \end{bmatrix}^T) = \begin{bmatrix} 1 & 0 \end{bmatrix}^T$$

$$f(0,1) = 1$$

It has learned correctly all 4 inputs!

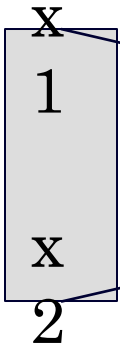
1 hidden layer Relu

Message: A network with 1 hidden layer and linear activations cannot learn the XOR function.

In fact a linear network with ANY number of hidden layers cannot learn the XOR function (since any number of layers still keeps it a linear function).

But a SINGLE hidden layer with Relu activations can learn the XOR function as this example shows.

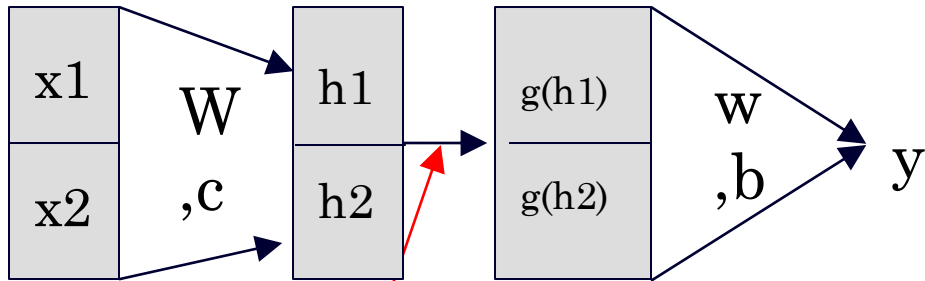
Furthermore networks with only one hidden layer and Relu activations can approximately learn ANY function (but hidden layer can be exponentially wide in input dimension).



Relu Link
function

See example from Ch6 GoodFellow et al. Deep Learning Book:
<https://www.deeplearningbook.org/contents/mlp.html>

Can you steal my network?



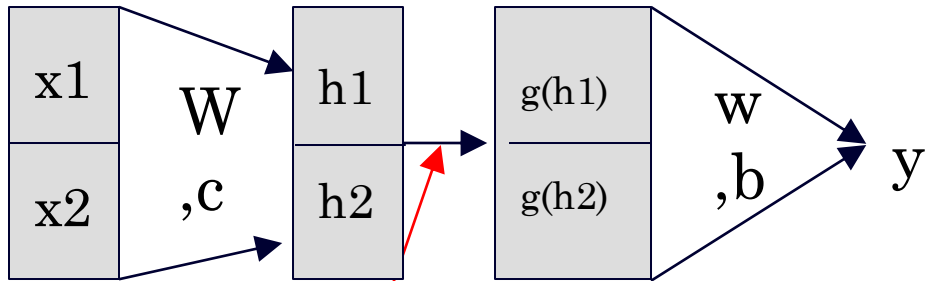
$g(z) = \max(z, 0)$
ReLU Activation function

$$\mathbf{h} = g(W^T \mathbf{x} + \mathbf{c})$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

Try $W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ $\mathbf{c} = [0, -1]^T$
 $\mathbf{w} = [1, -2]^T$ and $b = 0$

Stealing weights: Can you find some other $W_{\text{new}}, \mathbf{c}_{\text{new}}, \mathbf{w}_{\text{new}}, b_{\text{new}}$
So that output is the same for any input $\mathbf{x}$?

Can you steal my network?



$g(z) = \max(z, 0)$
ReLU Activation function

$$\mathbf{h} = g(W^T \mathbf{x} + \mathbf{c})$$
$$y = \mathbf{w}^T \mathbf{h} + b$$

Try $W = [1, 1; 1, 1]$ $\mathbf{c} = [0, -1]^T$
 $\mathbf{w} = [1, -2]^T$ and $b = 0$

Stealing weights: Can you find some other $W_{\text{new}}, \mathbf{c}_{\text{new}}, \mathbf{w}_{\text{new}}, b_{\text{new}}$
So that output is the same for any input $\mathbf{x}$?

Checking for stealing: Can you check if they compute the same output for any input?

Watermarking your weights: Can you check if they compute the same output for any input?

Backdoors: Can you modify the weights so that W_{new} is the same as W original but responds differently to only a specific any input?

Building & Training Neural Networks in PyTorch

- Limitations of Fixed Basis Functions
- Neural Networks
 - Neurons
 - Feed-Forward Neural Network
- Universality Theorem
- Activation Functions
- Building & Training Neural Networks in PyTorch



The most widely used **framework** for **building** and **training** deep **neural networks**.

- *Related frameworks*: TensorFlow, JaX, ...

Important features:

- **Automatic differentiation** (focus of Thursday's lecture)
- Intuitive support for **hardware accelerators**
- **Easy to use APIs** for state-of-the-art **model architectures**

Leaning on your own:

- Basics: <https://docs.pytorch.org/tutorials/beginner/basics>
- Tutorials: <https://docs.pytorch.org/tutorials/>

Big Concepts in PyTorch

- **torch.tensor** is the basic datatype used to encode data, weights, and activations and is very similar `numpy.ndarrays`.
- Neural networks are built using the many primitives in the **torch.nn** module and defining the **forward** process of making a prediction.
- PyTorch uses automatic differentiation to efficiently compute the **gradient**.
- Training is accomplished by **(you) implementing** some variation of gradient descent.

torch.tensor

The basic building block of PyTorch is the **torch.tensor** which is very similar to the **numpy.ndarray**.

- **Construction:** `torch.tensor([[1.0], [2.0]]);`
`torch.ones(2,3)`
- **Indexing and slicing:** `x[:, 3]`
- **Shape and type:** `x.shape; x.dtype`
- **Operations:** `torch.mean(x, axis=1); a.T @ x`

Key differences:

- **Device Placement:** `x.to("cuda"); x.to("cpu")`
- Tensors (can) track how they were computed to enable **automatic differentiation**: `x.grad_fn`

torch.nn.functional and torch.nn

The **torch.nn.functional** Python module All contains all the basic **stateless** neural network functions (e.g., relu).

The **torch.nn** Python module contains the **basic building blocks** of neural networks.

- **nn.Module** is the base class which all models and layers extend.
 - **__init__**: defines all the model parameters.
 - **forward** (**__call__**): defines the model/layer logic
- **nn.Parameters** is used to define model parameters (in **__init__**)
 - Used to wrap parameter tensors so they can be automatically tracked
- There are hundreds of layer modules: **nn.Linear**; **nn.Softmax**; ...

torch.nn.Module

When building a new model you extend the nn.Module

```
class MLPModel(nn.Module):
```

Extend nn.Module

```
    def __init__(self, dims, activation="relu"):
        super().__init__()
        self.layers = nn.ModuleList(
            [nn.Linear(dims[i], dims[i+1]) for i in range(len(dims)-1)])
        if activation == "relu": self.act = nn.functional.relu
        elif: self.act = torch.tanh

    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.layers)-1:
                x = self.act(x)

        return x
```

Add modules to your module. These add parameters.

Compute the **forward** pass using the layer modules.

Computing Gradients using Autograd

PyTorch uses **automatic differentiation** to **automatically** (and **efficiently**) **compute the gradient** of the loss with respect to all parameters.

```
model.zero_grad()
```

Reset the Gradient
(needed for iteration)

```
pred = model(x)
```

```
loss = loss_fn(pred, t)
```

Compute the loss

```
loss.backward()
```

Compute the **gradient** of the loss

```
print(model.linear.weight.grad)
```

Gradients are attached
to each parameter

Lecture 14

Neural Networks and PyTorch

Reference Book Chapters: Chapter 6 up to section 6.3.1